
DESIGNING AND EVALUATING A HAND GESTURE POINTER CONTROLL SYSTEM ON COMMON DESKTOP TASKS

A REPORT SUBMITTED AS A REQUIREMENT FOR EECS 255: ADVANCED HUMAN-COMPUTER INTERACTION

Po-Yun Cheng

University of California, Merced

December 13, 2025

ABSTRACT

Touchless input techniques based on hand gesture recognition (HGR) have received increasing attention as alternatives to traditional input devices, particularly in situations where physical contact is undesirable. In this project, we design a gesture-controlled pointer system based on MediaPipe hand landmarks, combined with a rule-based mapping algorithm to translate hand motions into pointer events. We conduct a user study comparing the proposed system with a touchpad across common desktop tasks, including menu selection, drag-and-drop, and virtual keyboard input. Quantitative results show that our system performs significantly worse than the touchpad in terms of task completion time and efficiency. Subjective feedback further indicates low usability and substantial physical demand during prolonged use. While the proposed system failed to be an acceptable input solution, this project serves as an exploratory investigation into the limitations of mid-air gesture-based desktop interaction, including large inter-user variability in gesture execution, and physical fatigue caused by sustained arm elevation. These findings offer insights to inform future research on more practical and ergonomic touchless interaction techniques.

1 Goals and Motivations

Touchless control [1] enables users to operate computer systems without physical contact, which helps alleviate hygiene or safety concerns on public shared surfaces, such as library computers. Thanks to the evolution of neural networks, webcam-based Hand Gesture Recognition (HGR) has become a popular technique for building touchless control methods [2, 3], as it is easy and lightweight to deploy without additional hardware, making it convenient to use in daily scenarios. Among all devices, desktop computers and laptops are the most frequently used interfaces for user interaction. Based on this background, we focus our project on Touchless Desktop Control using HGR techniques.

While many works have introduced touchless desktop control systems through HGR, most provide only pre-defined gesture mappings to fixed actions [4, 5, 6], which do not offer full control comparable to traditional input devices (e.g., touchpads). In this project, we develop a pilot system that aims to provide full desktop control through a gesture-controlled pointer. We further compare its performance with a traditional input device, the touchpad, across a set of designed tasks to evaluate the limitations of the proposed system in daily computer usage scenarios.

2 Design Process

To develop the gesture control system, we first designed four gestures that correspond to standard mouse pointer events, as shown in Figure 1. The gesture design follows the metaphor of traditional mouse usage, in which the thumb functions as a “virtual button” for the index and middle fingers. Additionally, the use of an open palm is assigned to pointer movement so that motion control does not interfere with button-event gestures.

The system is aimed to work on a standard laptop with a built-in webcam. While developing a custom recognition model could potentially improve performance and generalization, it would require substantial data collection and training. Instead, we use MediaPipe’s hand-landmark model, which is widely adopted in research and provides adequate

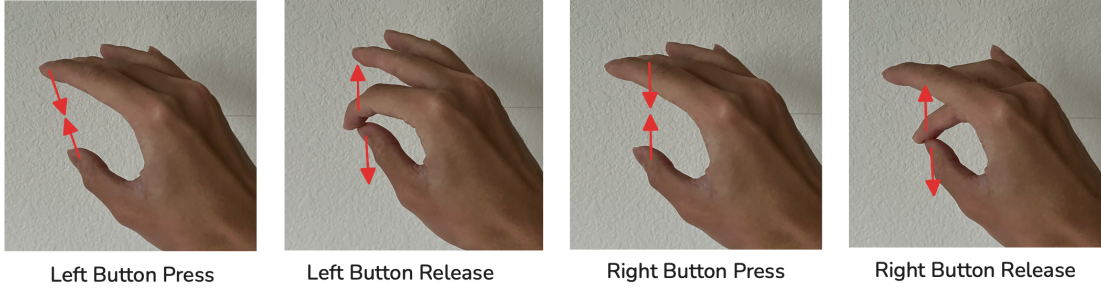


Figure 1: The four dynamic gestures representing different button actions. Pinching the index finger and thumb generates a left-button press, and releasing them generates a left-button release. The same applies to the right button, substituting the index finger with the middle finger.

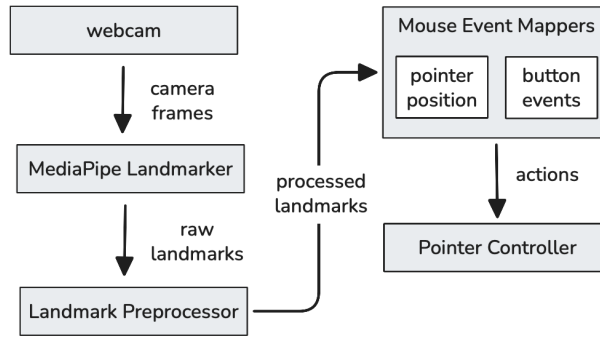


Figure 2: System architecture and data flow.

performance for validating our concept. A brief pilot study was conducted to refine the algorithm and determine key parameters.

The overall system architecture is shown in Figure 2. Camera frames are first processed by MediaPipe’s hand-landmark model to extract 3D hand-joint positions. A rule-based mapping algorithm then converts these positions into pointer-event representations. Subsequently, these events are sent to a native OS-level pointer control module to minimize latency. In parallel, a lightweight Python application provides real-time visual feedback to the user regarding the system’s status.

3 Implementation

The system was implemented in Python. We used the OpenCV library for camera access, MediaPipe for hand landmark detection, pyobjc-framework-Quartz for macOS pointer control, and PySide6 for the user interface. The system was executed on a 14-inch M4 MacBook Pro running macOS Sequoia 15.6.1. The camera frame rate was set to 30 FPS.

3.1 Pointer Position Mapping Algorithm

MediaPipe’s hand landmark model converts each camera frame into 21 hand landmarks with 3D coordinates (x, y, z) , where x and y represent normalized pixel coordinates relative to the top-left corner of the input frame, and z is an estimated depth value in world coordinates (with smaller values indicating points closer to the camera). To improve the stability of the detected landmark positions and reduce jitter, a preprocessing pipeline consisting of jump detection, temporal smoothing, and stationary detection was applied, as illustrated in Algorithm 1.

After preprocessing, the on-screen pointer position was computed following the steps illustrated in Figure 3. First, the positions of the WRIST, INDEX_MCP, and PINKY_MCP landmarks were used to compute the center point of the palm, denoted as point A . Next, the palm normal vector was obtained by computing the cross product between the vectors from A to INDEX_MCP and from A to PINKY_MCP. Finally, the intersection between the palm normal vector and an

Algorithm 1 Landmark Preprocess**Require:** smoothing factor $\alpha = 0.5$ **Require:** max window length $W = 5$ **Require:** jump threshold $\tau_{\text{jump}} = 0.001$, stationary threshold $\tau_{\text{stationary}} = 0.001$

```

1:  $L_{\text{prev}} \leftarrow \text{None}$ , window  $\leftarrow []$ 
2: function UPDATE( $L$ )  $\triangleright L$  is an array of shape  $21 \times 3$  contains positions of current detected landmarks
3:    $v \leftarrow \|L - L_{\text{prev}}\|_2$ 
4:   if  $v > \tau_{\text{jump}}$  then
5:     return  $L_{\text{prev}}$   $\triangleright$  Jump detected
6:   end if
7:    $L_{\text{smoothed}} \leftarrow \alpha L + (1 - \alpha)L_{\text{prev}}$   $\triangleright$  Exponential smoothing
8:
9:   Append  $L_{\text{smoothed}}$  to window  $\triangleright$  Update window
10:  if  $|\text{window}| > W$  then
11:    Remove oldest element from window
12:  end if
13:   $\bar{L} \leftarrow \text{mean}(\text{window})$ 
14:   $v_{\text{win}} \leftarrow \|L_{\text{smoothed}} - \bar{L}\|_2$ 
15:  if  $v_{\text{win}} < \tau_{\text{stationary}}$  then
16:    return ( $L_{\text{smoothed}}$ , True)  $\triangleright$  Stationary detected
17:  end if
18:  return ( $L_{\text{smoothed}}$ , False)
19: end function

```

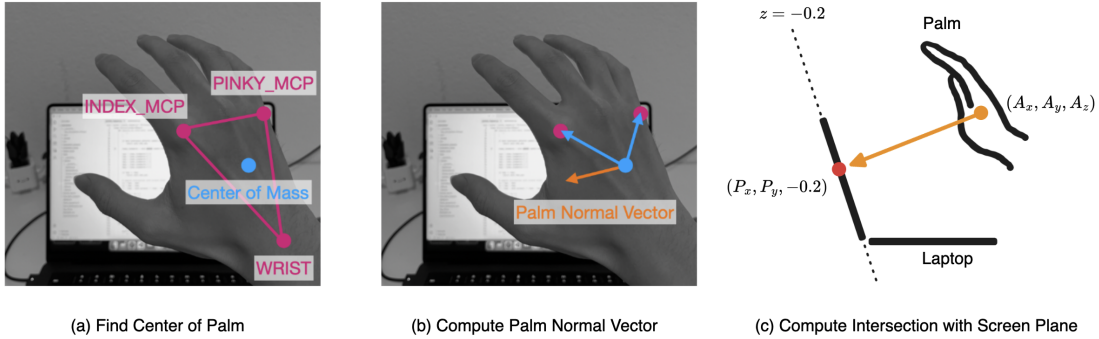


Figure 3: Steps of computing pointer position from hand landmarks.

estimated screen plane was calculated. The screen plane was defined as $z = -0.2$ in the landmark coordinate system. All parameters used in this mapping process were determined through manual calibration on pilot data.

3.2 Button Event Mapping Algorithm

To detect button press and release events, we maintain a sliding window of recent hand landmark positions with a fixed length of $M = 6$ frames. The window is temporally divided into two equal segments: a past window consisting of the oldest $M/2$ frames, and a current window consisting of the most recent $M/2$ frames. For each window segment, we compute the mean Euclidean distance between the thumb tip and the index finger tip, as well as between the thumb tip and the middle finger tip. The thumb–index distance is used to infer left-button events, while the thumb–middle distance corresponds to right-button events.

A press event is detected when the mean distance in the current window falls below a predefined threshold $\delta = 0.16$, while the corresponding distance in the past window remains above the threshold. Conversely, a release event is detected when the mean distance in the current window rises above the threshold while the past window distance is below it. If none of these conditions are met, no button event is triggered.

Since there are actually multiple frames that would be detected as press during a single pressing gesture as the gesture lasting more than one frame, therefore we only perform the detected event once we know the current event is ended,

that is, the next different event is detected, to avoid multiple events being triggered during a single gesture. That means the event is actually performed one frame delayed after the gesture over.

Another problem is that since we detect press and release event separately, which making a click event not atomic and noise or other events might occur between last frame of press and first frame of release. To prevent this, we use a threshold `CLICK_MERGE_SIZE` that we merge a press event and release event to a click event if they happened consecutively within the threshold. We set `CLICK_MERGE_SIZE` to 6 by computing the average frame of click in pilot data.

4 User Study

We conducted a user study to compare the performance of our proposed gesture control system with a traditional touchpad in common daily desktop interaction scenarios.

4.1 Apparatus

A custom Python program was developed to allow participants to perform the designed tasks. Both the study program and the gesture control system ran on a 14-inch M4 MacBook Pro. The built-in touchpad of the same device was used as the baseline input method for comparison. To evaluate subjective workload and usability, the NASA Task Load Index (NASA-TLX) and the System Usability Scale (SUS) were administered, respectively. In addition, a qualitative questionnaire was designed to collect participants' preferences and feedback.

4.2 Participants

Four participants (all male), aged between 20 and 30 years, were recruited for the study. All participants were right-handed.

4.3 Design

The study employed a within-subject design with two conditions: our system and a touchpad. Participants performed three tasks under both conditions. These tasks were designed to simulate common actions in daily desktop usage:

- **Menu Selection:** Participants right-clicked to open a context menu containing six options and then left-clicked the target option.
- **Drag and Drop:** Participants dragged a black square (left-click and hold) to a specified target area (A, B, C, or D) and released it to complete the task.
- **Keyboard Input:** Participants used a virtual keyboard to type a target word and submit it.

Each task consisted of five trials with randomized task parameters. Each trial could only be performed once and was recorded as correct, incorrect, or failed. The order of input conditions and tasks was counterbalanced using a Latin Square design.

4.4 Procedure

At the beginning of the study, participants were introduced to the task designs and received a brief tutorial on using our gesture control system in a desktop environment. Participants then performed the three tasks under both input conditions following the assigned order. Quantitative data such as correctness, completion time, and pointer moving distance is collecting for each trial. After completing each condition, participants filled out the NASA-TLX questionnaire. Upon completing both conditions, participants were asked to complete the SUS questionnaire and the qualitative feedback questionnaire.

4.5 Results

Figure 4 illustrates task efficiency and completion time for each trial. A task is considered more efficient when the task efficiency η is closer to 1, which is computed as follows:

$$\text{(Menu Selection, Drag Drop)} \quad \eta = \frac{\text{moving distance}}{\text{target distance}} \quad (1)$$

$$\text{(Keyboard Input)} \quad \eta = \frac{\text{number of key clicks}}{\text{len(target word)}} \quad (2)$$

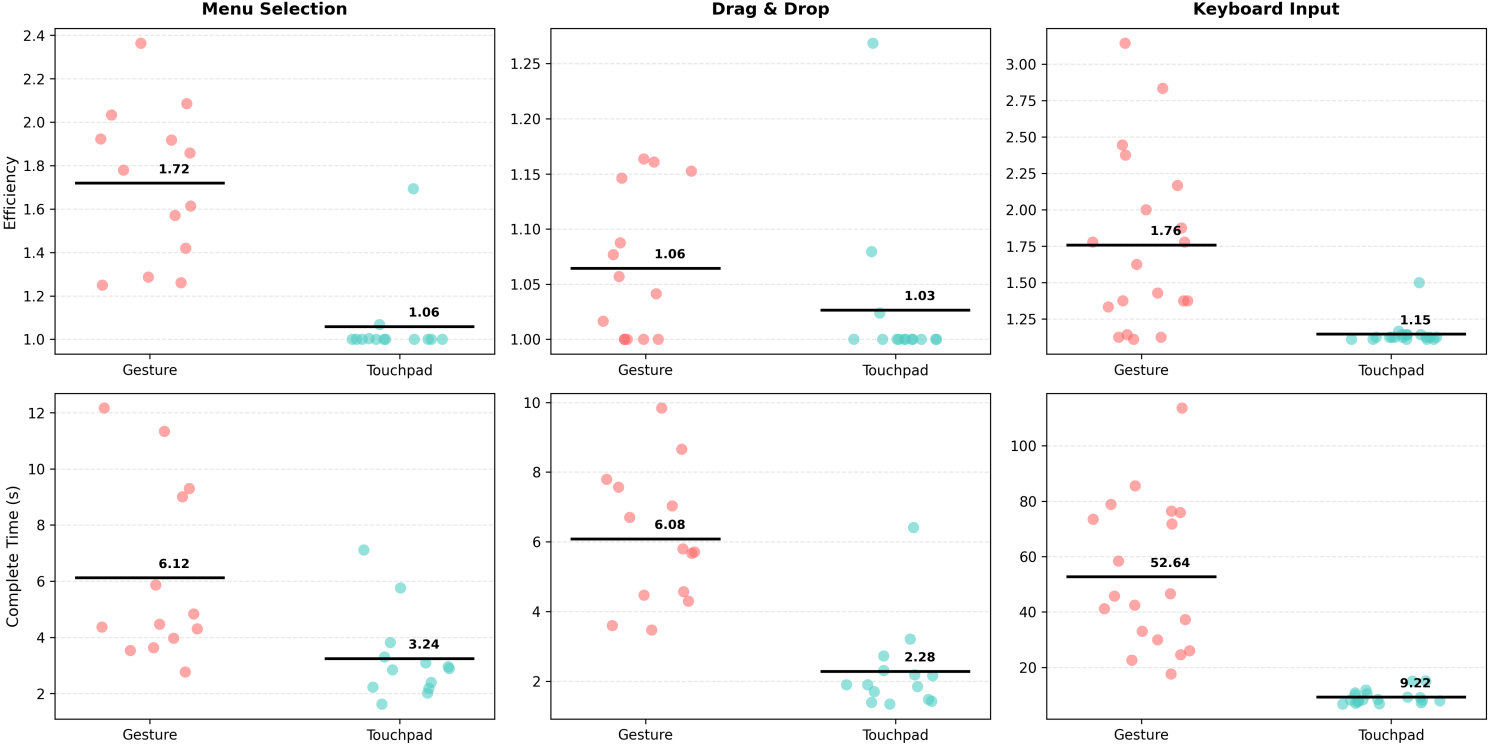


Figure 4: Task efficiency (top row) and completion time (bottom row) across tasks, comparing the gesture system (red) and the touchpad system (green).

For statistical analysis of task efficiency and completion time, we first conducted Shapiro-Wilk tests to assess normality. The results indicated that only the completion time of the drag-and-drop and keyboard input tasks, as well as the efficiency of the menu selection task, followed a normal distribution ($p > 0.05$). As most data did not satisfy the normality assumption, Wilcoxon signed-rank tests were applied. The results showed that, except for the efficiency of the drag-and-drop task ($p = 0.0745$), the gesture system performed significantly worse than the touchpad baseline ($p < 0.05$).

The overall task results, accuracy, and preference are summarized in Table 1. Accuracy was calculated as the number of correct trials divided by the total number of trials. Participant preference was obtained through a multiple-choice question in which participants selected their preferred input method among the tested systems.

The results of the NASA-TLX questionnaire are summarized in Table 2. Overall, the gesture system imposed a higher workload than the touchpad across tasks. In addition, the proposed system received a mean SUS score of 40.6, indicating that it was perceived as unacceptable in terms of usability.

Regarding qualitative feedback, all participants reported that holding their hand in mid-air caused fatigue. Additional feedback included: "The gesture system is not intuitive, and I am often unsure which gesture to perform to complete the task." (P1), and "The gesture system is not accurate and requires multiple attempts to trigger button clicks." (P2).

5 Discussion

The user study results suggest that the proposed system, in its current form, is not sufficiently refined to support daily desktop control as a fully usable input method. Rather than serving as a conclusive solution, the system functions as an exploratory prototype that reveals several fundamental challenges of mid-air gesture-based interaction.

One key observation from the project is that even when gestures are explicitly defined, significant variation exists in how different users perform the same gesture. Differences in motion range, execution speed, and movement style were consistently observed across participants. This variability exposes a limitation of static, rule-based thresholding, which

Task	System	Correct (trials)	Incorrect (trials)	Failed (trials)	Accuracy	Preference (person)
Menu Selection	gesture	11	2	7	55%	1
	touchpad	20	0	0	100%	3
Drag Drop	gesture	14	0	6	70%	1
	touchpad	20	0	0	100%	3
Keyboard Input	gesture	19	0	1	95%	0
	touchpad	20	0	0	100%	4

Table 1: Task results with accuracy and user preference. For the menu selection task, *incorrect* indicates selecting an incorrect option, and *failed* indicates clicking outside the menu. For the drag-and-drop task, *incorrect* indicates dropping the object in an incorrect target area, and *failed* indicates dropping it outside all target areas. For the keyboard input task, *incorrect* indicates entering an incorrect word, and *failed* indicates submitting no input.

	Frustration	Effort	Performance	Temporal Demand	Physical Demand	Mental Demand	Overall
gesture	53.8	81.2	48.8	38.8	70.0	65.0	59.6
touchpad	8.8	21.2	27.5	22.5	25.0	18.0	20.6

Table 2: NASA-TLX results for the gesture system and the touchpad system. Lower scores indicate lower perceived workload on the corresponding metric.

lacks the flexibility to accommodate individual differences. While user-specific calibration could potentially alleviate this issue, it would also introduce additional setup costs and reduce the immediacy of use.

Another potential approach to mitigating gesture variability is to reduce the system’s degrees of freedom. By constraining the dimension of action space or simplifying the gestures mapping, the recognition problem becomes more tractable and less sensitive to individual differences. However, this simplification potentially limits the range of interactions compared to traditional input devices. Future designs should carefully balance interaction richness with recognition stability.

Beyond recognition accuracy, physical demand emerged as a primary limitation of the proposed gesture control system. Unlike touch-based input devices that provide arm or wrist support, mid-air gesture interaction requires users to keep their hands raised in front of the camera for extended periods. This lack of physical support leads to rapid fatigue, as consistently reported by participants in the user study. The issue of physical demand highlights an inherent challenge of gesture-based interaction and suggests that gesture control may be better suited for short-term or intermittent interactions rather than prolonged continuous use.

6 Conclusion

In this project, we designed and implemented a touchless pointer control system using hand gesture recognition and rule-based mapping algorithm. Through a comparative user study against a touchpad, we evaluated the system’s performance in representative daily desktop tasks. While the results indicate that the proposed system is not good enough to be a replacement for conventional input devices. Across most tasks, the gesture system exhibited lower efficiency, longer completion times, and substantially higher perceived workload. Both quantitative measurements and subjective feedback consistently point to usability limitations and physical fatigue as major obstacles. Overall, this project highlights the challenges and trade-offs involved in designing gesture-based desktop input systems.

References

- [1] Muhammad Zahid Iqbal and Abraham G. Campbell. From luxury to necessity: Progress of touchless interaction technology. *Technology in Society*, 67:101796, 2021.

- [2] Shailaja Uke, Atifa Shaikh, Harshada Rayate, Anjali Kamble, and Sanavi Rahane. Towards touchless interaction: Implementing hand gesture recognition for presentation and media control. In *2025 International Conference on Emerging Smart Computing and Informatics (ESCI)*, pages 1–6, 2025.
- [3] Robert Noparlik and Rafał Zdunek. Hand gesture recognition system with finite state machine for remote desktop control. In *2023 8th International Conference on Smart and Sustainable Technologies (SpliTech)*, pages 1–6, 2023.
- [4] Sushma Khatri, Leeladhar Chourasia, Ankit Jain, and Yamini Barge. Touchless control: Hand-based gesture recognition for human-computer interaction. In *2024 Parul International Conference on Engineering and Technology (PICET)*, pages 1–6, 2024.
- [5] Zarinabegam Mundargi, Shreya Das, Aryan Shinde, Om Deokar, Shravani Bahirat, and Dipasha Shetiya. Hand gesture desktop control with python. In *2024 2nd International Conference on Advances in Computation, Communication and Information Technology (ICAICCIT)*, volume 1, pages 35–40, 2024.
- [6] Sriram S K and Nishant Sinha. Gestop: Customizable gesture control of computer systems. In *Proceedings of the 3rd ACM India Joint International Conference on Data Science & Management of Data (8th ACM IKDD CODS & 26th COMAD)*, CODS-COMAD '21, page 405–409, New York, NY, USA, 2021. Association for Computing Machinery.